

TAMANNA

[Github](#) | [LinkedIn](#) | [Leetcode](#)

tamannas10110@gmail.com | +91-9877553894

AI Engineer with 5+ years designing and delivering production-grade Gen AI applications across cloud and enterprise environments — from architecture and prototyping through deployment and production operations. Specialized in building agentic AI systems and retrieval-augmented generation (RAG) workflows for enterprise document management, including document ingestion, processing, classification, and retrieval, while maintaining responsible AI practices and data security/compliance standards. Proficient in Python and React, with strong grounding in Gen AI frameworks (LangChain, LangGraph), prompt and context engineering, and software engineering best practices across the development lifecycle. Experienced integrating AI platforms with enterprise SaaS ecosystems via secure API-driven architectures, and working independently with general guidance to deliver scalable, reliable solutions adopted across large enterprise user bases.

WORK EXPERIENCE

Genpact (Applied AI ML Engineer)	Bangalore, Karnataka July 2024 - Present
• Owned end-to-end technical direction for an agentic AI capability (Genpact AP Trace) — architecture, delivery, and reliability — embedding with cross-functional teams (finance, compliance, operations) to map workflows; shipped a multi-agent system (LangChain/LangGraph) flagging duplicates across 10,000+ invoices/month, reducing financial leakage risk by 15–20%. • Built multi-agent orchestration workflows (LangGraph) with state management, tool-calling, and structured outputs for vendor onboarding and compliance validation — designing guardrails and human-in-the-loop checkpoints for production LLM workflows (Azure OpenAI) — cutting manual processing effort by 40%. • Implemented compensating-control-style validation checks for GenAI document-processing pipelines (Azure OpenAI, Azure Document Intelligence) — monitoring output quality and triaging failures post-launch, achieving 90%+ extraction accuracy across 500+ samples. • Built RAG-grounded workflows with explainable, auditable outputs — debugging failure modes and improving reliability in production systems used by real end users, in a compliance-sensitive enterprise context. • Shipped a production Text-to-SQL service (Python, FastAPI, PostgreSQL) — applying secure-by-design practices for data access — adopted by 200+ users across 10+ regions, reducing manual SQL dependency by 60%. • Designed reusable, policy-driven orchestration patterns for a shared agentic AI framework across regions, with consistent interfaces and governance-aligned templates — accelerating onboarding by ~35%, cutting rollout time from 6 weeks to under 4. • Built secure tool connectors integrating enterprise systems (ServiceNow, Salesforce) via authenticated REST APIs with auditability and traceability in mind; applied security-first, production-grade code review practices.	
CGI (Software Engineer)	Bangalore, Karnataka June 2021 – June 2024
• Owned backend services end-to-end — design, implementation, deployment, and operational health — for cloud-native enterprise applications (React.js, Python — Flask/FastAPI), supporting 200+ enterprise users. • Designed REST API architectures (FastAPI, PostgreSQL, Redis) with strong backend development fundamentals, focusing on reliability and performance — reducing average API response time by 35%. • Built Python-based automations and third-party integrations, converting recurring operational issues into engineered, maintainable workflow solutions that replaced manual, legacy enterprise processes — reducing processing effort by 40–50% and eliminating 15–20 hrs/week of manual work. • Developed reusable React component libraries and coding standards, raising code quality and consistency across the team's solutions — cutting feature delivery time by 30%. • Built Selenium-based automation scripts for critical enterprise workflows, reducing failures by 30%; maintained technical documentation, used source control, and participated in code reviews for collaborative development and debugging. • Operated as a fully independent practitioner across the full application lifecycle with minimal supervision — requirements, architecture, build, and deployment — applying strong debugging instincts across legacy systems and ambiguous environments. • Integrated with third-party APIs as part of internal automation tooling, using PostgreSQL and Redis for backend data storage and caching needs. • Acted as a technical interface to business stakeholders, leading requirements gathering and demos, and clearly communicating technical concepts to non-technical audiences across cross-functional teams; worked across Linux and Windows environments using Python and command-line scripting for builds, deployments, and debugging.	

EDUCATION

Qualification	Institute	CGPA / Percentage	Time of Completion
Bachelor of Technology	NIT Jalandhar	CGPA – 7.92/10	August 2017 – May 2021
Class XII	GGN Public School, Ludhiana	91.8 %	April 2016 – March 2017
Class X	GGN Public School, Ludhiana	CGPA – 10/10	April 2014 – March 2015

TECHNICAL SKILLS

- **AI / GenAI / Agentic AI:** LLMs, Generative AI, Agentic AI, Multi-Agent Orchestration, LangChain, LangGraph, RAG, Prompt Engineering, Azure OpenAI, Azure Document Intelligence, Vector Databases (FAISS/Pinecone/ChromaDB), Function/Tool Calling, Scikit-Learn, TensorFlow, LLM finetuning using PEFT, RLHF
- **Languages:** Python, JavaScript, SQL, C++
- **Frontend:** React.js, Redux Toolkit, MUI (Material-UI), Tailwind CSS, HTML5/CSS3
- **Backend & APIs:** FastAPI, Flask, REST APIs, WebSockets, Microservices Architecture
- **Databases & Caching:** PostgreSQL, Redis, MongoDB
- **Cloud & DevOps:** Azure (App Service, Functions, OpenAI Service), Docker, Git, CI/CD
- **Data & Visualization:** NumPy, Pandas, Matplotlib, Seaborn
- **Automation & Testing:** Selenium, PyTest
- **CS Fundamentals:** Data Structures & Algorithms, Object-Oriented Programming, System Design

PERSONAL PROJECT

Agentic Medical Appointments Processing – [DEMO](#)

The system simulates how medical appointments move through an AI-orchestrated pipeline:

- **Orchestrator** — A master agent reads appointments from a database and ranks them by dynamic priority (client, specialty, notes).
- **Stage agents** — Each appointment passes through 6 configurable stages (e.g. Intake, Insurance, Clinical Review). Each stage is an LLM-backed agent that returns a standardized outcome.
- **Exception queue** — If a stage returns Escalate, the case goes to a human concierge.
- **Human resolution** — A human resolves the issue; the appointment becomes Cleared and the workflow resumes automatically.

Tech Stack Used – React, Vite, FastAPI, Python, SQLite, LangChain, LangGraph, Hugging Face LLM

CERTIFICATIONS

- [Document AI: From OCR to Agentic Doc Extraction](#) – May 2026
- [Databricks Certified Generative AI Engineer Associate](#)– July 2025